# Experiment-4

**Aim: Implement CPU scheduling algorithms (a) Round Robin (b)Pre-emptive priority job scheduling**

**a)Round Robin**

**Description:** Round Robin is a fair, preemptive job scheduling algorithm used by operating systems. It gives each process a fixed slice of time, called a time quantum, in a cyclic order. If a job does not finish in that time, the system pauses it and moves it to the back of the queue.

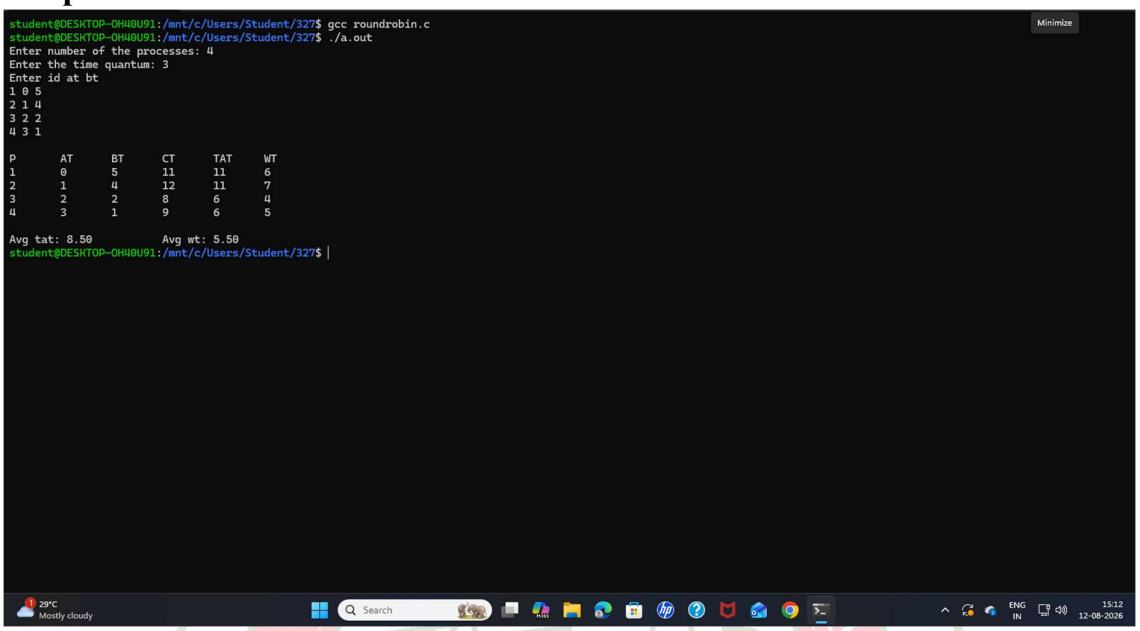
**Algorithm:**

1. Start
2. Read the number of processes n.
3. Read the time quantum q.
4. For each process, read:
   - Process ID (id)
   - Arrival Time (at)
   - Burst Time (bt)
5. Set:
   - rt = bt → Remaining Time
   - t = 0 → Current CPU time
   - a = 0 → Number of completed processes
6. Repeat while a < n:
   - Set b = 0.
   - Traverse all processes.
   - For every process satisfying:
     - at <= t
     - rt > 0
   - Mark CPU as busy: b = 1.
   - If rt > q:
     - Execute process for q units.
     - rt = rt - q
     - t = t + q
   - Otherwise:
     - Execute process until completion.
     - t = t + rt
     - Set rt = 0.
     - Calculate:
       - CT = t
       - TAT = CT - AT
       - WT = TAT - BT
     - Add TAT and WT to their respective totals.
     - Increment completed process count a.
7. If no process is available (b == 0), increment t by 1.
8. Calculate:
   - Average TAT = Total TAT / n
   - Average WT = Total WT / n
9. Display ID, AT, BT, CT, TAT, WT.
10. Stop

**Code:**

```
#include<stdio.h>
struct process {
int id;
int at;
int bt;
int ct;
int tat;
int wt;
int rt;
};
int main(){
int n,q,t=0,a=0,ttat=0,twt=0;
printf("Enter number of the processes: ");
scanf("%d", &n);
printf("Enter the time quantum: ");
scanf("%d" , &q);
struct process p[n];
printf("Enter id at bt\n");
for(int i=0; i<n ; i++){
scanf("%d %d %d" , &p[i].id,&p[i].at,&p[i].bt);
p[i].rt = p[i].bt;
}
while(a<n){
int b=0;
for(int i=0;i<n;i++){
if (p[i].at <= t && p[i].rt > 0) {
b = 1;
if(p[i].rt > q){
p[i].rt -=q;
t += q;
}
else{
t+= p[i].rt;
p[i].rt=0;
p[i].ct = t;
p[i].tat = p[i].ct - p[i].at;
p[i].wt = p[i].tat - p[i].bt;
ttat += p[i].tat;
twt += p[i].wt;
a++;
}
}
}
if(b == 0){
t++;
}
}
printf("\nP\tAT\tBT\tCT\tTAT\tWT\n");
for(int i =0; i<n;i++){
printf("%d\t%d\t%d\t%d\t%d\t%d\t",
p[i].id,p[i].at,p[i].bt,p[i].ct,p[i].tat,p[i].wt);
printf("\n");
}
printf("\nAvg tat: %.2f\t\tAvg wt: %.2f\n", ttat/(n*1.0),twt/(n*1.0));
return 0;
}
```

## Output:




### b)Preemptive priority job scheduling

**Description:** Preemptive priority job scheduling is an operating system method where the CPU immediately stops a running task if a new task with a higher priority arrives. The system saves the current task's state, runs the important new task first, and lets the old task finish later.

**Algorithm:**

1. Start
2. Read the number of processes n.
3. For each process, read:
   - Process ID (id)
   - Priority (p)
   - Arrival Time (at)
   - Burst Time (bt)
4. Set rt = bt for every process, where rt is the remaining burst time.
5. Initialize:
   - t = 0 → current time
   - completed = 0 → number of completed processes
   - ttat = 0 → total turnaround time
   - twt = 0 → total waiting time
6. Repeat while completed < n:
   - Set idx = -1.
   - Check all processes.
   - Select only processes satisfying:
     1. AT <= current time
     2. RT > 0
   - Among the available processes:
   - Select the process with the highest priority.
   - Here, smaller priority number means higher priority.
   - If two processes have the same priority:

- Select the process with the smaller remaining time.

7. If no process is available:
   - Increment t by 1.
   - Continue to the next iteration.
8. If a process is selected:
   - Execute it for 1 unit of CPU time.
   - Decrease its remaining time:
     RT = RT - 1
   - Increase current time:
     t = t + 1
9. If RT == 0, the process is completed:
   - Calculate Completion Time:
     CT = t
   - Calculate Turnaround Time:
     TAT = CT - AT
   - Calculate Waiting Time:
     WT = TAT - BT
   - Add TAT and WT to their respective totals.
   - Increment completed.
10. After all processes are completed, calculate:
    Average TAT = Total TAT / n
    Average WT = Total WT / n
11. Display the process details and averages.
12. Stop

**Code:**

```
#include <stdio.h>
struct process {
    int id, at, bt, ct, tat, wt, rt, p;
};
int main() {
    int n, t = 0, completed = 0;
    int ttat = 0, twt = 0;
    printf("Enter the number of processes: ");
    scanf("%d", &n);
    struct process p[n];
    printf("Enter process's id priority at bt:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d %d %d %d",&p[i].id,&p[i].p,&p[i].at,&p[i].bt);
        p[i].rt = p[i].bt;
    }
    while (completed < n) {
        int idx = -1;
        for (int i = 0; i < n; i++) {
            if (p[i].at <= t && p[i].rt > 0) {
                if (idx == -1 || p[i].p < p[idx].p) {
                    idx = i;
                }
                else if (p[i].p == p[idx].p &&
                         p[i].rt < p[idx].rt) {
                    idx = i;
                }
            }
        }
        if (idx == -1) {
```

```
            t++;
            continue;
        }
        p[idx].rt--;
        t++;
        if (p[idx].rt == 0) {
            p[idx].ct = t;
            p[idx].tat = p[idx].ct - p[idx].at;
            p[idx].wt = p[idx].tat - p[idx].bt;
            ttat += p[idx].tat;
            twt += p[idx].wt;
            completed++;
        }
    }
    printf("\nPID\tP\tAT\tBT\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",p[i].id,p[i].p,p[i].at,p[i].bt,p
[i].ct,p[i].tat,p[i].wt);
    }
    printf("\nAverage TAT = %.2f", (float)ttat / n);
    printf("\nAverage WT = %.2f\n", (float)twt / n);
    return 0;
}
```

**Output:**

